编译技术实验课

主讲教师：陈安龙

实验二：递归下降分析实验

1. 学习所提供的“表达式文法”的递归下降处理

   理解 rdlex.l、rdparser.c的内容

2. 学习理解sysy文法

   理解 sysy.y 方法各产生式含义 （sysy.y含有左递归和公共左因子）

   尝试将语言定义文档中方法转换为递归下降可用文法

   将自己转化的文法与 sysy.y 文法相比较

3. 编写sysy.y 文法的递归下降程序

sysy文法

```
CompUnit: Decl CompUnit
        | FuncDef CompUnit
        | Decl
        | FuncDef

Decl: ConstDecl
    | VarDecl

ConstDecl: Y_CONST Type ConstDef Y_SEMICOLON
         | Y_CONST Type ConstDefs Y_SEMICOLON

ConstDefs: ConstDef Y_COMMA ConstDef
         | ConstDefs Y_COMMA ConstDef

ConstDef: Y_ID Y_ASSIGN ConstInitVal
        | Y_ID ConstExps Y_ASSIGN ConstInitVal

ConstExps: Y_LSQUARE ConstExp Y_RSQUARE
         | Y_LSQUARE ConstExp Y_RSQUARE ConstExps
```

```
ConstInitVal: ConstExp
            | Y_LBRACKET Y_RBRACKET
            | Y_LBRACKET ConstInitVal Y_RBRACKET
            | Y_LBRACKET ConstInitVal ConstInitVals Y_RBRACKET

ConstInitVals: Y_COMMA ConstInitVal
             | Y_COMMA ConstInitVal ConstInitVals

VarDecl: Type VarDef Y_SEMICOLON
       | Type VarDef VarDecls Y_SEMICOLON

VarDecls: Y_COMMA VarDef
        | Y_COMMA VarDef VarDecls

VarDef: Y_ID
      | Y_ID Y_ASSIGN InitVal
      | Y_ID ConstExps
      | Y_ID ConstExps Y_ASSIGN InitVal
```

sysy文法(续)

```
29 InitVal: Exp
30        | Y_LBRACKET Y_RBRACKET
31        | Y_LBRACKET InitVal Y_RBRACKET
32        | Y_LBRACKET InitVal InitVals Y_RBRACKET

33 InitVals: Y_COMMA InitVal
34         | Y_COMMA InitVal InitVals

35 FuncDef: Type Y_ID Y_LPAR Y_RPAR Block
36        | Type Y_ID Y_LPAR FuncParams Y_RPAR Block

37 FuncParams: FuncParam
38           | FuncParams Y_COMMA FuncParam

39 FuncParam: Type Y_ID
40          | Type Y_ID Y_LSQUARE Y_RSQUARE
41          | Type Y_ID ArraySubscripts
42          | Type Y_ID Y_LSQUARE Y_RSQUARE ArraySubscripts
```

```
43 Block: Y_LBRACKET BlockItems Y_RBRACKET
44      | Y_LBRACKET Y_RBRACKET

45 BlockItems: BlockItem
46           | BlockItem BlockItems

47 BlockItem: Decl
48          | Stmt

49 Stmt: LVal Y_ASSIGN Exp Y_SEMICOLON
50     | Y_SEMICOLON
51     | Exp Y_SEMICOLON
52     | Block
53     | Y_WHILE Y_LPAR LOrExp Y_RPAR Stmt
54     | Y_IF Y_LPAR LOrExp Y_RPAR Stmt
55     | Y_IF Y_LPAR LOrExp Y_RPAR Stmt Y_ELSE Stmt
56     | Y_BREAK Y_SEMICOLON
57     | Y_CONTINUE Y_SEMICOLON
58     | Y_RETURN Exp Y_SEMICOLON
59     | Y_RETURN Y_SEMICOLON
```

sysy文法（续）

```
60 Exp: AddExp

61 LVal: Y_ID
62     | Y_ID ArraySubscripts

63 ArraySubscripts: Y_LSQUARE Exp Y_RSQUARE
64                | Y_LSQUARE Exp Y_RSQUARE ArraySubscripts

65 PrimaryExp: Y_LPAR Exp Y_RPAR
66           | LVal
67           | num_INT
68           | num_FLOAT

69 UnaryExp: PrimaryExp
70         | Y_ID Y_LPAR Y_RPAR
71         | Y_ID Y_LPAR CallParams Y_RPAR
72         | Y_ADD UnaryExp
73         | Y_SUB UnaryExp
74         | Y_NOT UnaryExp

75 CallParams: Exp
76           | Exp Y_COMMA CallParams

77 MulExp: UnaryExp
78       | MulExp Y_MUL UnaryExp
79       | MulExp Y_DIV UnaryExp
80       | MulExp Y_MODULO UnaryExp
```

```
81 AddExp: MulExp
82       | AddExp Y_ADD MulExp
83       | AddExp Y_SUB MulExp

84 RelExp: AddExp
85       | AddExp Y_LESS RelExp
86       | AddExp Y_GREAT RelExp
87       | AddExp Y_LESSEQ RelExp
88       | AddExp Y_GREATEQ RelExp

89 EqExp: RelExp
90      | RelExp Y_EQ EqExp
91      | RelExp Y_NOTEQ EqExp

92 LAndExp: EqExp
93        | EqExp Y_AND LAndExp

94 LOrExp: LAndExp
95       | LAndExp Y_OR LOrExp

96 ConstExp: AddExp

97 Type: Y_INT
98     | Y_FLOAT
99     | Y_VOID
```

实验任务

（1）根据实验所给文法，完成非终结符 CallParams、RelExp、Stmt对应的递归下降函数。

past rd_relexp();

past rd_call_paras();

past rd_stmt();

实验任务

（2）其中非终结符 Block 和 ArraySubscripts 对应的递归下降函数已实现，可直接调用，函数的声明如下：

```
past rd_block();
past rd_array_subscripts();
```

实验任务

（3）词法分析的设定如下：

```
// 单词类别
enum yytokentype {
        num_INT = 258,
        num_FLOAT = 259,
        Y_ID = 260,
        ...
};
```

```
typedef struct _TokenType{
        enum yytokentype token; // 单词类别
        union {
                int             ivalue;
                float   fvalue;
                char*   svalue;
        }attr; // 单词属性
}TokenType;
```

实验任务

（4）下列函数的功能：

```
extern TokenType cur_token;//全局变量，表示当前的单词
TokenType advance(); //读取下一个单词，将其保存于 cur_token 中，同时返回该单词
// 下面两个函数用于可能的回溯处理，即如果有多个分支需要尝试，在尝试语法分析前，可调用get_cur_tok_index获取当前单词的位置；
// 之后在分析时可能会调用若干次advance函数，如果分析失败，则可通过set_cur_tok_index函数重置当前单词的位置。
int get_cur_tok_index(); //所有单词已经读入单词数组中，该函数获取当前单词在数组中的下标
void set_cur_tok_index(int ind); //将 cur_token 设置为下标指定的token值
```

实验任务

（5）抽象语法树的设定如下：

```
typedef struct _ast ast;
typedef struct _ast *past;
struct _ast{
  int ivalue;
  float fvalue;
  char* svalue;
  node_type nodeType;
  past left;
  past right;
  past if_cond;
  past next;
};
```

```
past newAstNode();
// 创建一个结点
past newID(char* value);
//创建一个结点，并将结点中的 svalue 设置为 value
past newInt(int value);
//创建一个结点，并将结点中的 ivalue 设置为 value
```

实验安排要求

1、在icoding系统完成代码编写调试，并提交系统；

2、每位同学需一份实验报告的word文档，并在系统中上传；

3、所有文本文件都以 utf-8 进行统一编码保存！！！

4、请同学注意系统的提交截止时间，无论任何原因在截止前没有提交，一切后果需要同学自己负责，老师无权修改截止日期。

递归下降分析举例

```
exp: factor
        | exp '+' factor
        | exp '-' factor
        ;
factor: term
        | factor '*' term
        | factor '/' term
        ;
term: NUMBER
        | '-' term
        ;
```

递归下降分析举例

```
exp: factor
       | exp '+' factor
       | exp '-' factor
       ;
```

语言：factor +/- factor +/- factor ...

程序：

```
void exp(){
  factor();

  while( tok == '+' || tok == '-'){
      advance();
      factor();
  }
}
```

+/- factor +/- factor ...
重复出现的部分

递归下降分析举例

```
factor: term
      | factor '*' term
      | factor '/' term
      ;
```

语言：term */ term */ term ...

*/ term */ term ...
重复出现的部分

程序：

```
void factor(){
  term();

  while( tok == '*' || tok == '/'){
      advance();
      term();
  }
}
```

递归下降分析举例

```
term: NUMBER
        | '-' term
        ;
```

语言：NUMBER、 - NUMBER
- - NUMBER、- - - NUMBER

程序：

```
void term(){
  if( tok == NUMBER );
     advance();
  else if ( tok == '-'){
      advance();
      term();
  }
  else error();
}
```

递归下降分析举例

纯粹的语法分析只分析

词法记号序列是否符合语言的文法

如： -5+20*5-3

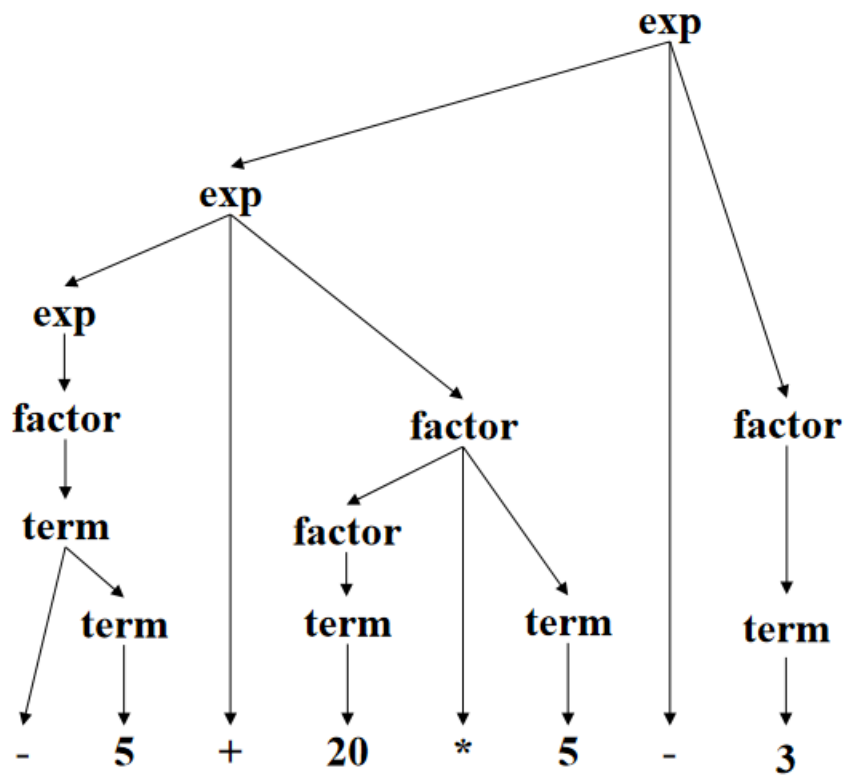
实际的编译器

需要找出词法记号序列的结构

即语法树

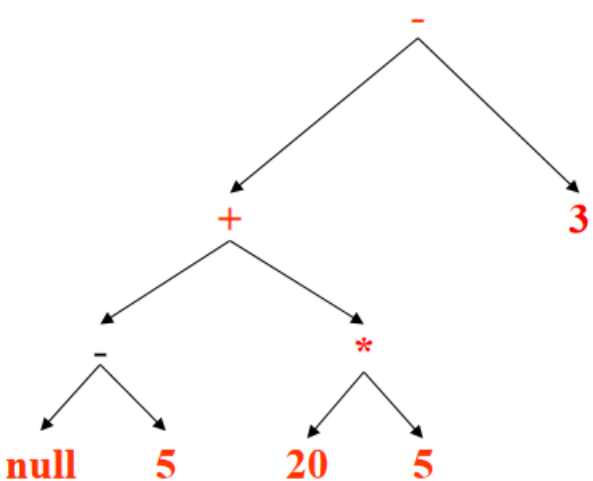
递归下降文法举例

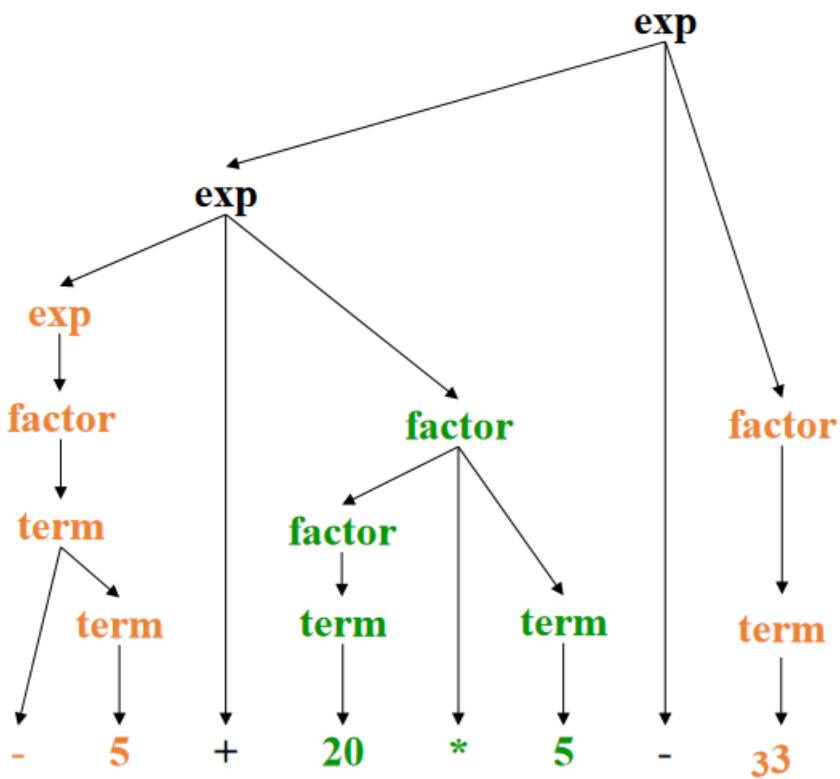
```
exp: factor
        | exp '+' factor
        | exp '-' factor
        ;
factor: term
        | factor '*' term
        | factor '/' term
        ;
term: NUMBER
        | '-' term
        ;
```



递归下降文法举例

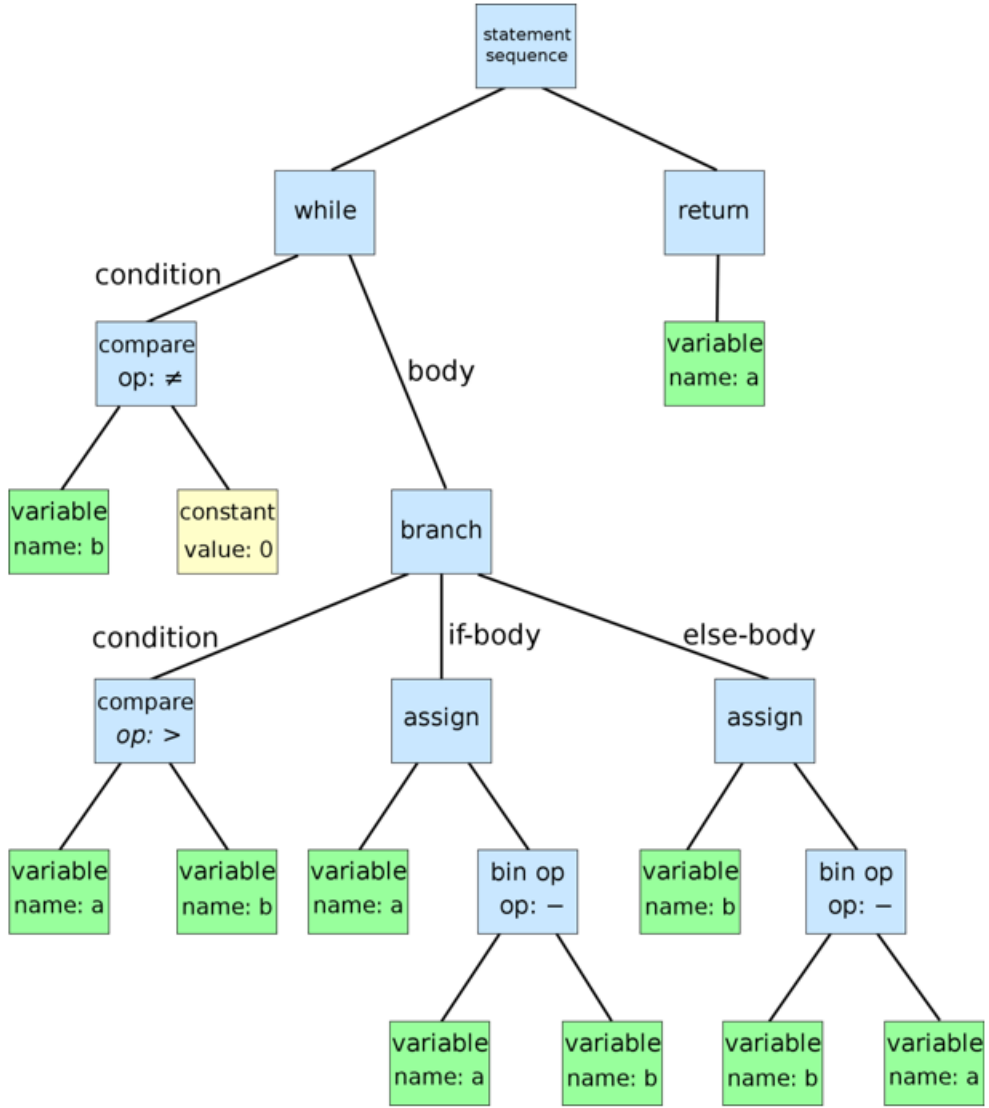
抽象的语法树
（Abstract Syntax Tree, AST）





抽象语法树

```
while b ≠ 0
    if a > b
        a := a − b
    else
        b := b − a
return a
```



表达式的抽象语法树

抽象语法树结点设计

- 所有树结点统一为 一个 结构类型
- 如何区别结点：赋值语句？选择语句？
  - 结点内需要 用标志位
- 结点的最大分支数
  - 表达式最多为二元运算，至少 2 个
- 结点为树的叶结点
  - 需要保存运算的数值

表达式的抽象语法树

抽象语法树结点设计

```
typedef struct _ast ast;
typedef struct _ast *past;
struct _ast{
        int ivalue;
        float fvalue;
        char* svalue;
        node_type nodeType;
        past left;
        past right;
        past if_cond;
        past next;
};
```

node_Type 必须使用 note_type.h 中定义的类型

并列关系必须由 next 指针串联

if_cond 仅限于 if 语句中的条件

表达式的抽象语法树

递归下降分析函数

```
void factor(){
  term();
  while( tok == '*' || tok == '/')
  {
     advance();
     term();
  }
}
```

```
past astFactor()
{
        past l = astTerm();
        while(tok == '*' || tok == '/')
        {
                int oper = tok;
                advance();
                past r = astTerm();
                l = newExpr(oper, l, r);
        }
        return l;
}
```